



数据结构 期末考试复习总结

闭卷笔试 | 120分钟 | 占总成绩50%

第一章 绪论

核心概念

概念	说明
数据元素	数据的基本单位，在程序中作为一个整体处理
数据结构	相互之间存在一种或多种特定关系的数据元素的集合
逻辑结构	数据元素之间的逻辑关系（集合、线性、树形、图状）
物理结构（存储结构）	数据结构在计算机中的存储表示（顺序存储、链式存储）
抽象数据类型 ADT	数学模型 + 定义在该模型上的一组操作

算法分析

度量	含义
时间复杂度	算法执行时间随问题规模增长的趋势，用 O 表示
空间复杂度	算法所需存储空间的量度
最坏/平均/最好	三种常见的时间复杂度分析场景

⚡ 常见时间复杂度排序

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

第二章 线性表

定义

有限个同类型数据元素组成的**有限序列**。除第一个元素外每个元素有唯一前驱，除最后一个外每个元素有唯一后继。

两种存储方式对比

特性	顺序表	单链表
存储方式	连续内存单元	任意位置，指针链接
存取方式	随机存取 $O(1)$	顺序存取 $O(n)$
插入/删除	$O(n)$ （需移动元素）	$O(1)$ （改指针，已定位时）
空间	固定分配，可能浪费	动态分配，指针额外开销
按值查找	$O(n)$	$O(n)$

必会操作

- 顺序表：插入、删除（注意边界和移动方向）
- 单链表：头插法/尾插法建表、插入、删除、反转
- 双向链表：插入/删除需要修改前驱和后继指针
- 循环链表：判断空表条件（`L->next == L`）

第三章 栈和队列

栈（Stack） — 后进先出 LIFO

要点	说明
栈顶/栈底	插入删除在栈顶进行
顺序栈	用数组实现，注意栈满判断 <code>top == MAXSIZE-1</code>
链栈	用链表实现，无栈满问题

要点	说明
应用	括号匹配、表达式求值、递归、DFS、函数调用栈

队列（Queue） — 先进先出 FIFO

要点	说明
队头/队尾	删除在队头，插入在队尾
循环队列	解决假溢出， front 指向队头， rear 指向队尾下一位置
判空	front == rear
判满	(rear+1) % MAXSIZE == front （牺牲一个单元）
应用	BFS、缓冲区、打印机队列

第四章 串

基本概念

- **串**：零个或多个字符组成的有限序列
- **空串** vs **空格串**：空串长度为 0，空格串由一个或多个空格组成
- **子串**：串中任意连续字符组成的子序列

存储结构

- **顺序存储**：用字符数组
- **块链存储**：用链表，每个结点存多个字符

核心操作

- 字符串匹配算法（简单匹配 / KMP 了解思想）

第五章 数组和广义表

数组

- **特点**：结构固定，定义后维数和维界不变，只能存取/修改元素
- **二维数组地址计算**（行优先存储，每个元素占 L 字节）：

$$LOC(a_{ij}) = LOC(a_{00}) + (i \times n + j) \times L$$

特殊矩阵压缩

矩阵类型	压缩方式	存储元素数
对称矩阵	只存上三角或下三角	$n(n+1)/2$
三角矩阵	存三角 + 1个常量	$n(n+1)/2 + 1$
对角矩阵	按对角带存储	约 $(3d-2)n$
稀疏矩阵	三元组表示（行、列、值）	非零元素个数

广义表

- 每个元素可以是原子或子表
- 长度 = 最外层元素个数；深度 = 嵌套的最大层数

第六章 树和二叉树

基本术语

术语	含义
结点的 度	该结点拥有的子树数
树的 度	树中所有结点度的最大值
叶子结点	度为 0 的结点
深度/高度	树的最大层次

二叉树性质（必背！）

- 1. 第 i 层最多 2^{i-1} 个结点
- 2. 深度为 k 的二叉树最多 $2^k - 1$ 个结点
- 3. $n_0 = n_2 + 1$ （叶子数 = 度为2的结点数 + 1）
- 4. 具有 n 个结点的**完全二叉树**深度为 $\lfloor \log_2 n \rfloor + 1$
- 5. 完全二叉树中，结点 i 的左孩子为 $2i$ ，右孩子为 $2i + 1$ ，双亲为 $\lfloor i/2 \rfloor$

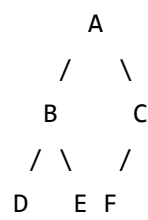
满二叉树 vs 完全二叉树

- **满二叉树**：深度为 k 且有 $2^k - 1$ 个结点，每层都满
- **完全二叉树**：除最后一层外每层满，最后一层结点靠左排列

二叉树的存储

- **顺序存储**：按完全二叉树编号存数组（适合满/完全二叉树）
- **链式存储**：二叉链表（lchild, data, rchild）

★ 四种遍历（递归/非递归都要会）



遍历方式	顺序	示例结果
先序	根→左→右	A B D E C F
中序	左→根→右	D B E A F C
后序	左→右→根	D E B F C A
层序	逐层从左到右	A B C D E F

🔑 **关键结论**：已知 **先序+中序** 或 **后序+中序** 可以唯一确定一棵二叉树。仅先序+后序不能！

线索二叉树

- 利用空的左右指针域存放**前驱/后继**线索

- 目的：加快遍历速度，无需递归/栈

哈夫曼树（Huffman Tree）

- 带权路径长度 $WPL = \sum(\text{权值} \times \text{路径长度})$
- 哈夫曼树：WPL 最小的二叉树
- 构建方法：每次选两个权值最小的结点合并
- 哈夫曼编码：左 0 右 1，前缀编码（任一编码不是其他编码的前缀）

树、森林 ↔ 二叉树的转换

- 树→二叉树：兄弟相连，保留左孩子（左孩子右兄弟）
- 森林→二叉树：每棵树转二叉树，再逐一接到前一树的右链
- 二叉树→树：逆过程

第七章 图

基本术语

术语	含义
完全图	无向图 $n(n-1)/2$ 条边；有向图 $n(n-1)$ 条弧
连通图	任意两顶点间有路径的无向图
强连通图	任意两顶点间有路径的有向图
生成树	包含全部顶点的极小连通子图（ n 个顶点， $n-1$ 条边）

存储结构

方式	空间	适用场景
邻接矩阵	$O(n^2)$	稠密图
邻接表	$O(n+e)$	稀疏图

图的遍历

- DFS（深度优先）：递归或栈实现，类似树的先序遍历
- BFS（广度优先）：队列实现，类似树的层序遍历

★ 最小生成树

算法	思想	时间复杂度
Prim	从点出发，每次选最小边加入	$O(n^2)$ ，适合稠密图
Kruskal	从边出发，每次选最小边（不形成回路）	$O(e \log e)$ ，适合稀疏图

AOV 网与拓扑排序

- AOV 网：顶点表示活动，有向边表示先后关系
- 拓扑排序：选无前驱的顶点输出，删除其出边，重复
- 可检测 DAG 是否存在环

最短路径

- Dijkstra：单源最短路径，按路径长度递增次序（不能处理负权边）
- Floyd：多源最短路径， $O(n^3)$

第九章 查找 🔥（薄弱！重点复习）

一、静态查找

顺序查找

项目	说明
适用存储	顺序或链式
适用状态	有序或无序均可
ASL成功	$(n + 1)/2$
ASL不成功	n 或 $n + 1$ （有哨兵时为 $n+1$ ）

项目	说明
时间复杂度	$O(n)$

折半查找（二分查找）

项目	说明
前提	⚠️ 顺序存储 且 有序
思想	每次折半，与中间值比较
判定树	平衡二叉树，深度 $\lfloor \log_2 n \rfloor + 1$
ASL成功	$\approx \log_2(n + 1) - 1$
时间复杂度	$O(\log n)$
适用	不常变动的有序表（插入删除代价大）

分块查找（索引顺序查找）

- 块内无序，块间有序；建立索引表
- $ASL = \text{索引查找ASL} + \text{块内查找ASL}$

二、二叉排序树 BST ★

定义

要么是空树，要么满足：

1. 左子树上所有结点值 < 根结点值
2. 右子树上所有结点值 > 根结点值
3. 左右子树也分别是二叉排序树

特点（必记！）

- 中序遍历得到有序（递增）序列
- 插入、删除后仍要保持 BST 性质
- 查找效率取决于树的高度，最好 $O(\log n)$ ，最坏 $O(n)$ （退化成链表）
- 不是平衡树，插入顺序决定形状

插入

- 与根比较，小去左，大去右，找到空位插入
- 新插入的结点一定是叶子结点

删除（3种情况）

情况	处理方法
叶子结点	直接删除
只有左子树或右子树	用孩子替代
有左右子树	用 中序前驱 （左子树最右）或 中序后继 （右子树最左）替代

三、平衡二叉树 AVL ★★

定义

空树，或满足：

1. 左右子树高度差绝对值 ≤ 1
 2. 左右子树都是 AVL 树
- **平衡因子** = 左子树高度 - 右子树高度，取值范围 $\{-1, 0, +1\}$

四种失衡 & 旋转（重中之重！）

失衡类型	特征	旋转方法
LL	左孩子的左子树插入导致	右旋 （顺时针转根）
RR	右孩子的右子树插入导致	左旋 （逆时针转根）
LR	左孩子的右子树插入导致	先左旋左子树，再右旋根
RL	右孩子的左子树插入导致	先右旋右子树，再左旋根

LL 示例（对 A 右旋）： RR 示例（对 A 左旋）：



AVL 插入步骤

- 按 BST 规则插入新结点
- 从插入点向上回溯，检查每个祖先的平衡因子
- 找到第一个失衡结点，判断类型（LL/RR/LR/RL）
- 执行对应旋转

四、B- 树和 B+ 树

B- 树（m 阶）

性质	根结点	非根非叶结点	叶子结点
最少关键字	1	$\lceil m/2 \rceil - 1$	0
最多关键字	m-1	m-1	—
最少子树	2	$\lceil m/2 \rceil$	无
最多子树	m	m	无

 B-树中所有叶子结点在同一层，且不包含信息（是查找失败的标志）。

B- 树插入（3阶为例）

- 3阶 B-树最多 **2** 个关键字，**3** 个子树
- 超过 2 个关键字时需要**分裂**：
 - 取中间关键字提升到父结点
 - 左右关键字分别形成两个新结点

B+ 树 vs B- 树（重要区分！）

区别	B- 树	B+ 树
关键字分布	每个结点都存关键字+记录	非叶结点只存索引，叶结点存全部记录

区别	B- 树	B+ 树
查找终止	可能在非叶结点找到	必须走到叶子才能确认找到
叶子链接	无	叶子结点按序链接，支持顺序查找
应用	文件索引	数据库索引（范围查询更高效）

五、散列表（哈希表）☆☆

核心术语

术语	含义
散列函数 $H(key)$	将关键字映射到地址的函数
同义词	具有相同散列地址的不同关键字
冲突	不同关键字映射到同一地址
装填因子 α	$\alpha = \text{表中记录数} / \text{表长}$ ， α 越大越容易冲突

常见散列函数

- 1. 除留余数法： $H(key) = key \text{ MOD } p$ ， p 取不大于表长的素数
- 2. 平方取中法、折叠法等

☆☆ 冲突处理方法

1. 开放定址法

$$H_i = (H(key) + d_i) \bmod m$$

方法	d_i 取值	特点
线性探测	1, 2, 3, ...	简单，但会产生聚集
二次探测	$\pm 1^2, \pm 2^2, \pm 3^2, \dots$	减少聚集，但表长应为素数
伪随机探测	随机序列	—

2. 链地址法（拉链法）

- 每个地址维护一个链表，同义词链在一起
- 不会产生聚集，但需要额外指针空间

- 装填因子可以 > 1

二次探测详解（重点！）

$$h_i(k) = (H(k) \pm i^2) \bmod m$$

以 $m=17$, $H(k)=k\%17$, 序列 {23,22,7,26,9,6} 为例：

key	H(key)	冲突?	探测过程	最终位置
23	6	无	—	6
22	5	无	—	5
7	7	无	—	7
26	9	无	—	9
9	9	 与26冲突	$(9+1)\%17=10$ ✓	10
6	6	 与23冲突	$(6+1)\%17=7$ (冲突) $\rightarrow(6-1)\%17=5$ (冲突) $\rightarrow(6+4)\%17=10$ (冲突) $\rightarrow(6-4)\%17=2$ ✓	2

 二次探测中， $\pm i^2$ 要交替尝试：+1, -1, +4, -4, +9, -9, ...

平均查找长度 ASL 计算

$ASL_{成功} = \text{各关键字查找比较次数之和} / \text{关键字个数}$

以线性探测为例：建表后，每个关键字的查找次数 = 从散列地址到实际地址经过的步数（包括自身）。

第十章 内部排序

★ 九大排序算法总览

排序算法	最好	平均	最坏	空间	稳定?	一趟定位置?
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	 稳定	
折半插入	$O(n \log n)$	$O(n^2)$	$O(n^2)$	$O(1)$	 稳定	

排序算法	最好	平均	最坏	空间	稳定?	一趟定位置?
希尔排序	$O(n^{1.3})$	—	$O(n^2)$	$O(1)$	✗	✗
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ 稳定	✓
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗	✓
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗	✓
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗	✓
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ 稳定	✗
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	✓ 稳定	—

核心辨析

- 不受初始状态影响的（时间复杂度恒定）：堆排、归并、简单选择、基数
- 原地排序 $O(1)$ 空间：插入、选择、冒泡、希尔、堆排
- 不稳定：选择、希尔、快排、堆排（选希快堆 → 记忆口诀："选修快对"）
- 一趟确定一个最终位置：选择、冒泡、快排、堆排

快速排序

- 枢轴 (pivot)：经过一趟划分后，枢轴左侧全 $\leq$ 它，右侧全 $\geq$ 它，枢轴到达最终位置
- 最坏情况：序列已正序或逆序（退化 $O(n^2)$ ）
- 最好情况：每次枢轴将序列均匀分成两半

堆排序

- 大顶堆：每个结点 $\geq$ 其孩子；小顶堆：每个结点 $\leq$ 其孩子
- 建堆：从 $\lfloor n/2 \rfloor$ 到 1 依次下调
- 排序：输出堆顶（与末尾交换），再调整

归并排序

- 2 路归并：将两个有序子序列合并成一个有序序列
- 空间复杂度 $O(n)$ （需要辅助数组）
- 最稳定的 $O(n \log n)$ 算法

如何选择排序算法？

场景	推荐
n 很小	直接插入 / 简单选择
基本有序	直接插入 / 冒泡
n 很大，要求稳定	归并排序
n 很大，要求快	快速排序
选前 k 个最大/最小	堆排序
要求 $O(n \log n)$ 且稳定	归并排序
多关键字	基数排序

附录：应试技巧

常见题型解题要点

1. **折半查找判定树**：画树，看查找路径
2. **二叉排序树构造**：按顺序插入，每次从根开始比较
3. **AVL 旋转判断**：先建树→找到失衡点→判断 LL/RR/LR/RL
4. **B-树插入**：记住 m 阶最多 m-1 个关键字，超过就分裂
5. **哈希表建表**：一步一步按冲突策略填，耐心计算
6. **排序过程识别**：看每一趟的特点（冒泡的最后面有序，快排的有枢轴居中，归并的局部有序）
7. **快速排序枢轴识别**：一趟后枢轴左侧都小于它，右侧都大于它

选填题高频陷阱

- 二叉排序树中序遍历得到有序序列 
- 快排空间复杂度 $O(\log n)$ （递归栈），不是 $O(1)$ 
- 折半查找要求**顺序存储+有序**，两个条件缺一不可
- B+ 树查找必须查到叶子，非叶结点匹配不等于找到
- 链地址法不会产生"聚集"（二次聚集）